Module: INTE 22313 : Software Design Patterns & Frameworks (24/25)
Assignment: In-Class Work (20/7/2026)
Index No: IM/2023/114

Automated B2B Credit Risk and Portfolio Management System

Contents

1. Project Overview
2. Use Case Identification
3. Singleton Pattern
4. Factory Method Pattern
5. Facade Pattern
6. Reflection and Comparison
7. Generative AI Usage

---

1. Project Overview

1.1 Project Description

The Automated B2B Credit Risk and Portfolio Management System automatically decides how much credit to extend to a B2B customer, tracks whether they pay on time, blocks them automatically if they do not, and gradually increases their limit the more they prove trustworthy, without a staff member having to track any of it by hand. It is being built for a wholesale paint manufacturing enterprise that currently manages hundreds of customer accounts through a manual ledger process.

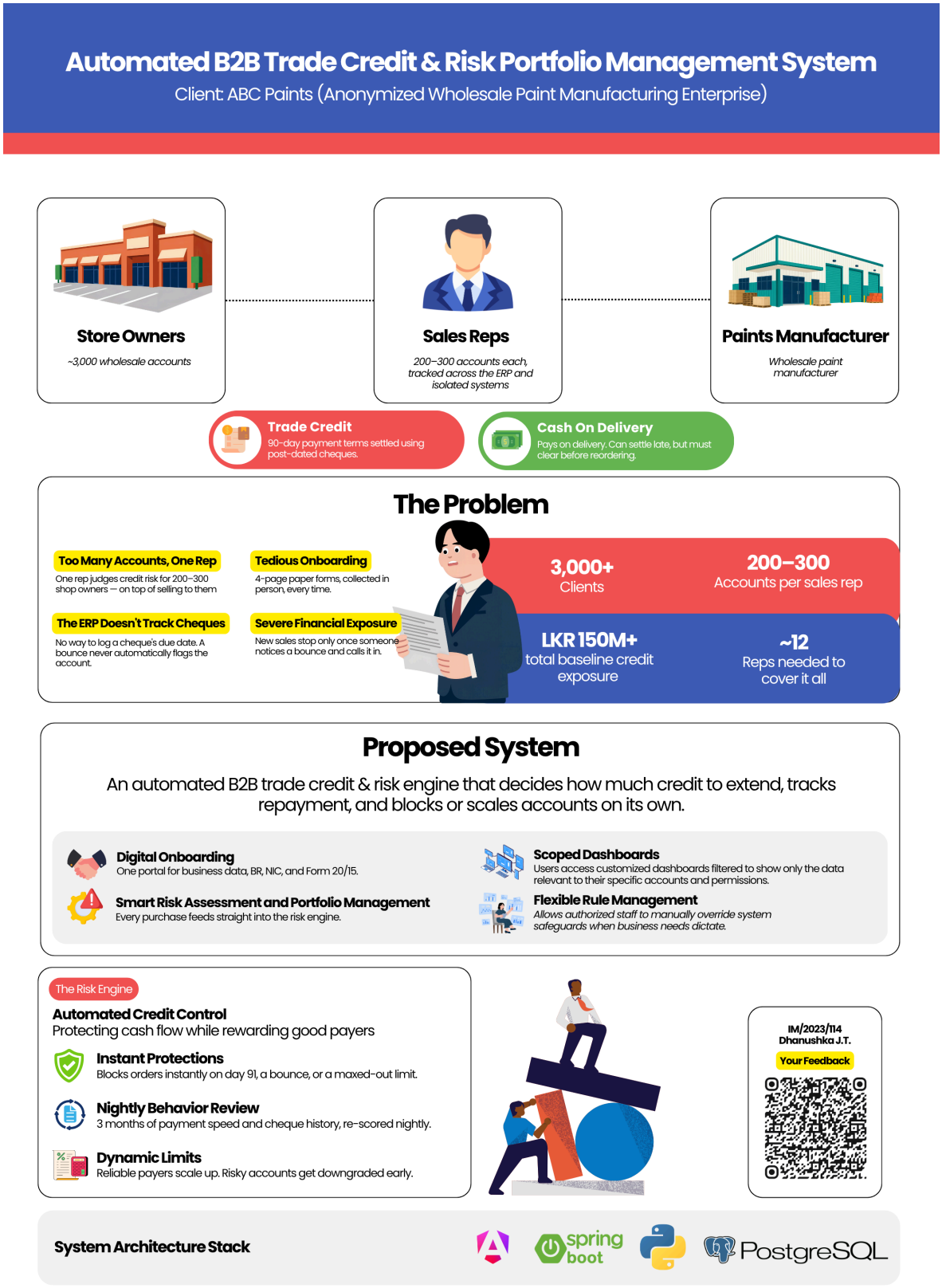
Customers fall into two types. Cash (COD) customers are assigned a due-amount threshold by back-office staff at onboarding. Before any new order can be placed, a Cash customer must have cleared their outstanding balance in full; if any balance remains, the order is blocked and an exception is raised to the Sales Manager or Director for review. Credit customers operate under the full credit-safeguard engine: a 90-day credit period, a credit-limit check, a 30-day post-dated cheque limit, and a nightly engine that recalculates their limit based on trailing 3-month behaviour. The system does not integrate a payment gateway. It only tracks amounts owed and updates them as back-office staff record settlements.

1.2 Problem the System Solves

In the current manual process, each sales representative tracks roughly 300 customer accounts using spreadsheets, checking ledgers for invoices approaching the 90-day mark and phoning management for limit exceptions. This does not scale: overdue invoices get missed, bounced cheques are not acted on fast enough, and every customer is stuck at a flat starting limit because nobody has time to manually reassess

hundreds of accounts. The system replaces this reactive manual ledger with a proactive, rule-driven engine so risk is caught the moment it happens and trustworthy customers are rewarded automatically.

1.3 Demonstrated Poster



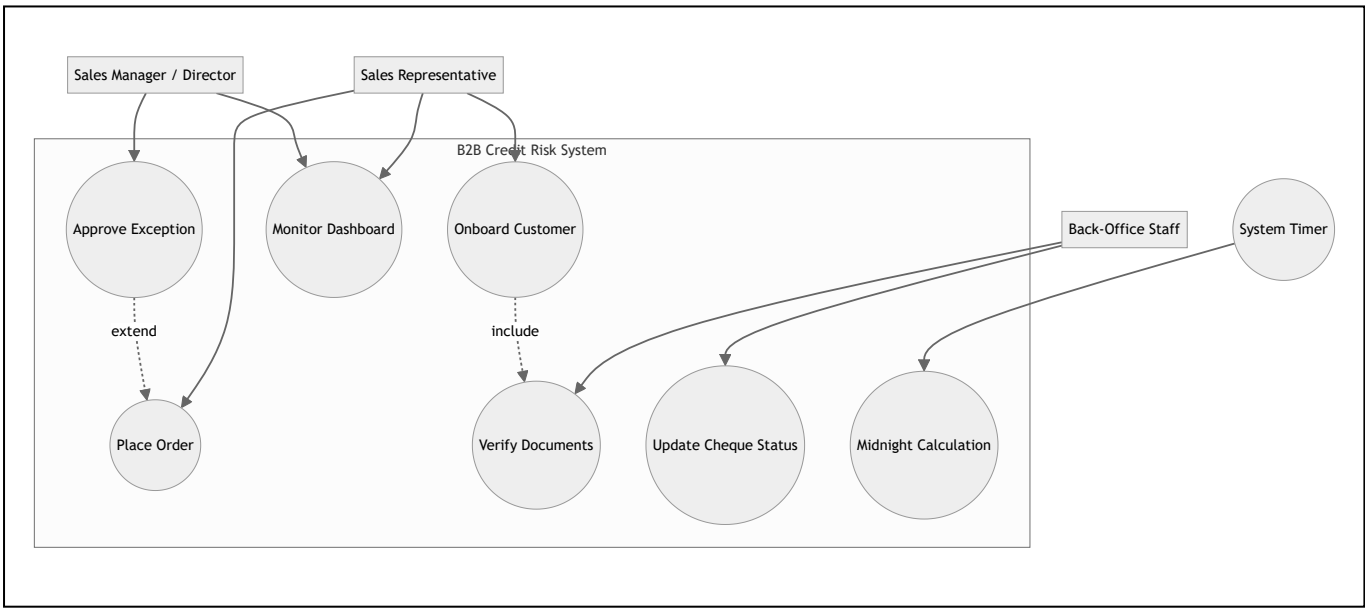
1.4 Main Users / Stakeholders

Stakeholder	Role in the System
Business Customer	The end client, either Cash (COD) or Credit type. Interacts with the system indirectly; a Sales Representative captures their details and places orders on their behalf during site visits.
Sales Representative	Onboards new customers in the field, places orders on behalf of assigned clients, and monitors a restricted view of their own customers' limit and block status.
Back-Office Staff	Verifies uploaded onboarding documents, assigns the due-amount threshold for Cash customers, and manages post-dated cheque status for Credit customers.
Sales Manager / Director (System Administrator)	Approves or rejects pending order exceptions for both Cash (COD) customers with an outstanding balance and Credit customers who have exceeded their credit limit. Has full visibility into portfolio risk on the administrator dashboard.
System Timer (automated actor)	Triggers the nightly Midnight Calculation batch job that rescales every active Credit customer's limit.

---

2. Use Case Identification

2.1 Use Case Diagram



2.2 Use Case Descriptions

Use Case	Actor(s)	Description

Onboard Customer	Sales Representative	Captures a new business customer's personal, business, and banking details and uploads identification and registration documents during a site visit, then presents the digital credit agreement for consent. Always includes document verification as a mandatory step.
Verify Customer Documents	Back-Office Staff	Reviews uploaded onboarding documents and either approves them, activating the customer's account and assigning a starting due threshold or credit limit, or rejects the submission with a documented reason.
Place Order on Behalf of Client	Sales Representative	Places an order for an assigned customer. Cash (COD) customers are assigned a due-amount threshold at onboarding and must clear their outstanding balance in full before any new order can be placed; if a balance remains, the order is blocked and an exception is raised to the Sales Manager or Director. Credit customers are checked against their credit limit and routed to Pending if it is breached. The system only tracks amounts owed; it does not process payments directly.
Update Cheque Status	Back-Office Staff	Logs post-dated cheque details against a Credit customer's order and updates its status to Pending, Cleared, or Bounced. Marking a cheque Bounced immediately revokes the customer's purchasing privileges.
Approve Order Exception	Sales Manager / Director	Reviews the order history and payment behaviour for any order placed into the Pending queue, whether it is a Cash (COD) customer who has an outstanding balance that has not been cleared, or a Credit customer who has exceeded their assigned credit limit, then approves or rejects it. Extends "Place Order on Behalf of Client" only when an exception condition is triggered.
Monitor Credit Dashboard / API	Sales Representative, Sales Manager / Director	Views portfolio risk data. Sales Representatives see a restricted view scoped to their own assigned customers; the Administrator sees full portfolio exposure, blocked accounts, and upcoming cheque dates.
Execute Midnight Calculation	System Timer	A scheduled batch job that runs nightly, analysing each active Credit customer's trailing 3-month purchase frequency, payment speed, and cheque clearance rate to automatically scale their credit limit up or down.

---

3. Applying the Singleton Design Pattern

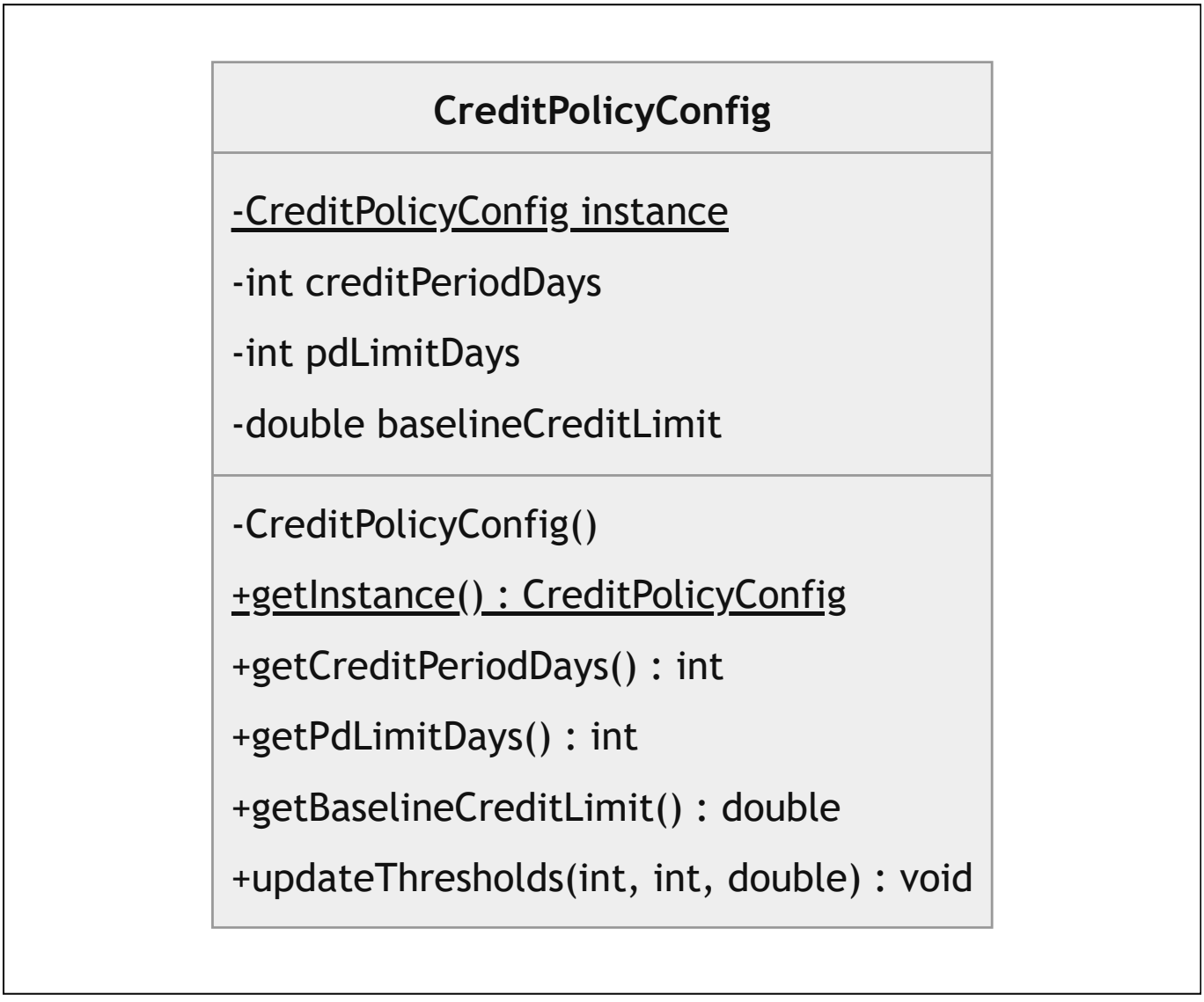
3.1 Scenario

The configurable thresholds used across the system, the 90-day credit period, the 30-day post-dated cheque limit, and the default baseline credit limit, must be read consistently by every part of the system that touches them: order validation, the nightly recalculation engine, and onboarding. A CreditPolicyConfig class holding these thresholds is a suitable candidate for the Singleton pattern.

3.2 Why Singleton Is Appropriate

- There must be exactly one authoritative copy of the threshold values in memory. Two independently loaded copies could drift out of sync and cause an order to be validated against stale rules.
- The configuration is read constantly, on every order placed and every nightly recalculation, but changed rarely, only when an administrator updates a threshold, so a single shared instance avoids repeated reloading.
- It gives the codebase one well-known access point, CreditPolicyConfig.getInstance(), instead of passing configuration objects through every layer of the application.

3.3 UML Class Diagram



3.4 Java Implementation

```
public class CreditPolicyConfig {

    private int creditPeriodDays;
    private int pdLimitDays;
    private double baselineCreditLimit;
```

```
    // Private constructor prevents external instantiation
    private CreditPolicyConfig() {
        // Defaults; in production these would be loaded from the database or config service
        this.creditPeriodDays = 90;
        this.pdLimitDays = 30;
        this.baselineCreditLimit = 50000.0;
    }

    // Bill Pugh holder idiom: lazy-loaded and thread-safe via the JVM classloader,
    // with no synchronization overhead on repeated access
    private static class Holder {
        private static final CreditPolicyConfig INSTANCE = new CreditPolicyConfig();
    }

    public static CreditPolicyConfig getInstance() {
        return Holder.INSTANCE;
    }

    public int getCreditPeriodDays() {
        return creditPeriodDays;
    }

    public int getPdLimitDays() {
        return pdLimitDays;
    }

    public double getBaselineCreditLimit() {
        return baselineCreditLimit;
    }

    // Lets an administrator update thresholds at runtime
    public synchronized void updateThresholds(int creditPeriodDays, int pdLimitDays, double
baselineCreditLimit) {
        this.creditPeriodDays = creditPeriodDays;
        this.pdLimitDays = pdLimitDays;
        this.baselineCreditLimit = baselineCreditLimit;
    }
}

// Usage anywhere in the codebase:
// int period = CreditPolicyConfig.getInstance().getCreditPeriodDays();
```

---

4. Applying the Factory Method Design Pattern

4.1 Scenario

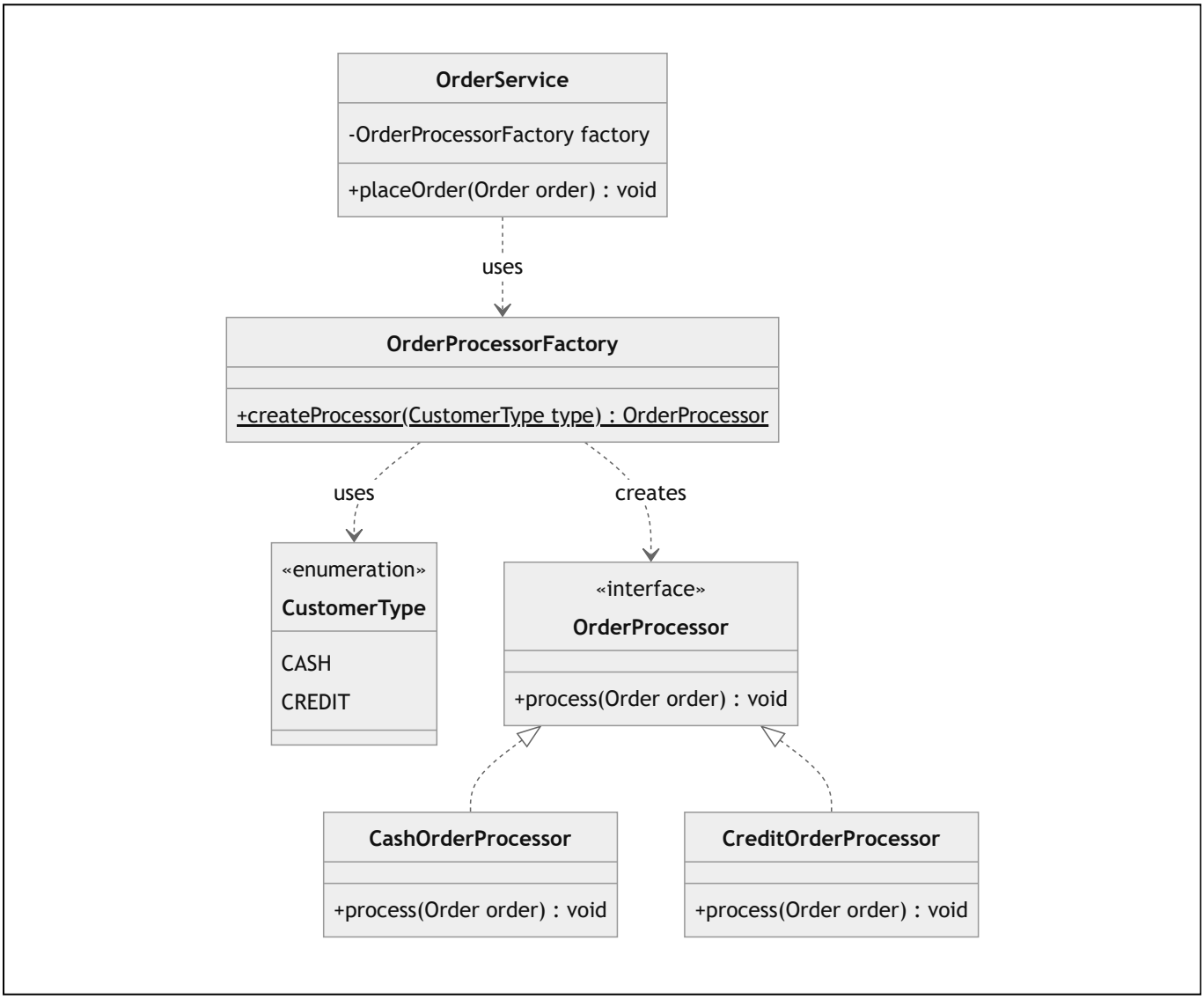
Every order is placed by either a Cash (COD) customer or a Credit customer, and each type needs different processing logic. Cash customers are assigned a due-amount threshold at onboarding. Before a new order can be processed, a Cash customer must have no outstanding balance; if any balance remains, the order is blocked and an exception is raised to the Sales Manager or Director. No payment gateway is

involved; the system only tracks what is owed until it is settled after delivery. Credit customers are checked against their credit limit and, if it is exceeded, routed to a Pending state for management approval. The order service should not need a long if/else chain to know how to handle each type.

4.2 Why Factory Method Is Suitable

- It decouples the client, the order service, from the concrete processor classes. The client only depends on the OrderProcessor interface.
- Adding a new customer type in the future means adding one new class and one new enum value, with no changes to any existing client code.
- Each concrete processor encapsulates its own rule set, keeping the business logic for each customer type isolated and independently testable.

4.3 UML Class Diagram



4.4 Java Implementation

```
public enum CustomerType {
    CASH, CREDIT
```

```
}

public enum OrderStatus {
    APPROVED, PENDING, BLOCKED
}

// --- Minimal supporting classes ---
public class Customer {
    private String id;
    private CustomerType customerType;
    private double outstandingBalance; // unpaid amount (Credit) or unsettled due amount
(Cash)
    private double assignedLimit;      // credit limit (Credit) or due-amount threshold (Cash)

    public CustomerType getCustomerType() { return customerType; }
    public double getOutstandingBalance() { return outstandingBalance; }
    public double getAssignedLimit() { return assignedLimit; }
    public void addOutstandingBalance(double amount) { this.outstandingBalance += amount; }
}

public class Order {
    private Customer customer;
    private double amount;
    private OrderStatus status;

    public Customer getCustomer() { return customer; }
    public double getAmount() { return amount; }
    public void setStatus(OrderStatus status) { this.status = status; }
}

// --- Product interface ---
public interface OrderProcessor {
    void process(Order order);
}

// --- Concrete products ---
public class CashOrderProcessor implements OrderProcessor {
    @Override
    public void process(Order order) {
        // COD customers must clear any outstanding balance before a new order is approved.
        // If a balance remains, the order is blocked and an exception is raised to
management.
        if (order.getCustomer().getOutstandingBalance() > 0) {
            order.setStatus(OrderStatus.PENDING); // escalated to Sales Manager / Director
        } else {
            order.setStatus(OrderStatus.APPROVED);
            order.getCustomer().addOutstandingBalance(order.getAmount());
        }
    }
}

public class CreditOrderProcessor implements OrderProcessor {
    @Override
    public void process(Order order) {
```

```
        double projectedBalance = order.getCustomer().getOutstandingBalance() +
order.getAmount();
        if (projectedBalance > order.getCustomer().getAssignedLimit()) {
            order.setStatus(OrderStatus.PENDING); // routed to the exception approval workflow
        } else {
            order.setStatus(OrderStatus.APPROVED);
            order.getCustomer().addOutstandingBalance(order.getAmount());
        }
    }
}

// --- Factory Method ---
public class OrderProcessorFactory {
    public static OrderProcessor createProcessor(CustomerType type) {
        switch (type) {
            case CASH:
                return new CashOrderProcessor();
            case CREDIT:
                return new CreditOrderProcessor();
            default:
                throw new IllegalArgumentException("Unsupported customer type: " + type);
        }
    }
}

// --- Client ---
public class OrderService {
    public void placeOrder(Order order) {
        OrderProcessor processor =
OrderProcessorFactory.createProcessor(order.getCustomer().getCustomerType());
        processor.process(order);
    }
}
```

---

5. Applying the Facade Design Pattern

5.1 Complex Subsystem

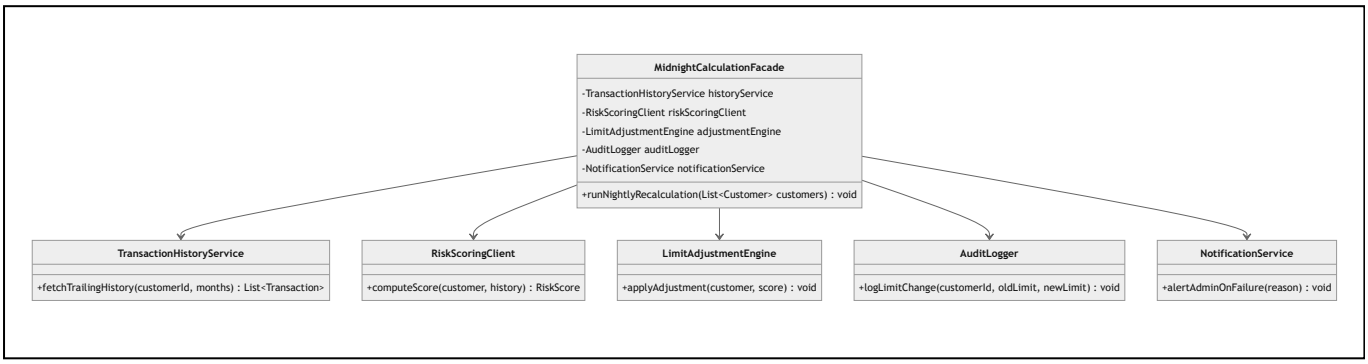
The nightly Midnight Calculation Engine is the most complex subsystem in the project. To rescale one Credit customer's limit, the system has to fetch their trailing 3-month transaction history, call the risk-scoring microservice to compute a behavioural score, apply the resulting adjustment to their limit, write an immutable audit log entry, and, if anything fails, alert the administrator and leave the prior day's limits untouched. That is five separate subsystems that the System Timer's cron trigger would otherwise have to orchestrate directly.

5.2 How the Facade Improves Usability

- The cron job calls a single method, runNightlyRecalculation(), instead of knowing about, sequencing, and error-handling five different services.

- It centralises the fail-safe behaviour in one place, so a failure partway through the batch cannot leave the system in an inconsistent state.
- The subsystems, history fetching, risk scoring, adjustment, logging, and notification, can each evolve independently, including the risk-scoring microservice being swapped or iterated on, without the caller needing to change at all.

5.3 UML Class Diagram



5.4 Java Implementation

```
import java.util.List;

// --- Subsystem classes (simplified) ---
public class TransactionHistoryService {
    public List<Transaction> fetchTrailingHistory(String customerId, int months) {
        return TransactionRepository.findByCustomerSince(customerId, months);
    }
}

public class RiskScoringClient {
    public RiskScore computeScore(Customer customer, List<Transaction> history) {
        // Calls the risk-scoring microservice (exponential decay algorithm)
        return RiskServiceGateway.score(customer, history);
    }
}

public class LimitAdjustmentEngine {
    public void applyAdjustment(Customer customer, RiskScore score) {
        double newLimit = score.recommendLimit(customer.getAssignedLimit());
        customer.setAssignedLimit(newLimit);
    }
}

public class AuditLogger {
    public void logLimitChange(String customerId, double oldLimit, double newLimit) {
        AuditRepository.save(new AuditEntry(customerId, oldLimit, newLimit));
    }
}

public class NotificationService {
    public void alertAdminOnFailure(String reason) {
        AdminAlertGateway.send("Midnight Calculation failed: " + reason);
```

```
    }
}

// --- Facade ---
public class MidnightCalculationFacade {

    private final TransactionHistoryService historyService;
    private final RiskScoringClient riskScoringClient;
    private final LimitAdjustmentEngine adjustmentEngine;
    private final AuditLogger auditLogger;
    private final NotificationService notificationService;

    public MidnightCalculationFacade() {
        this.historyService = new TransactionHistoryService();
        this.riskScoringClient = new RiskScoringClient();
        this.adjustmentEngine = new LimitAdjustmentEngine();
        this.auditLogger = new AuditLogger();
        this.notificationService = new NotificationService();
    }

    // Single entry point the cron job calls
    public void runNightlyRecalculation(List<Customer> activeCustomers) {
        for (Customer customer : activeCustomers) {
            double previousLimit = customer.getAssignedLimit();
            try {
                List<Transaction> history =
historyService.fetchTrailingHistory(customer.getId(), 3);
                RiskScore score = riskScoringClient.computeScore(customer, history);
                adjustmentEngine.applyAdjustment(customer, score);
                auditLogger.logLimitChange(customer.getId(), previousLimit,
customer.getAssignedLimit());
            } catch (Exception e) {
                // Fail-safe: keep the prior limit unchanged, alert the administrator
                customer.setAssignedLimit(previousLimit);
                notificationService.alertAdminOnFailure(e.getMessage());
            }
        }
    }
}

// --- Client (the scheduled job) ---
public class MidnightCalculationJob {
    private final MidnightCalculationFacade facade = new MidnightCalculationFacade();

    public void run() {
        List<Customer> activeCustomers = CustomerRepository.findAllActive();
        facade.runNightlyRecalculation(activeCustomers);
    }
}
```

6. Reflection and Comparison

6.1 Benefits in This Project

- Singleton (CreditPolicyConfig): guarantees every part of the system reads the same threshold values, avoids reloading configuration repeatedly, and gives a single point to update thresholds at runtime without redeploying.
- Factory Method (OrderProcessorFactory): keeps the order service free of customer-type conditionals, isolates each customer type's business rules in its own class, and makes adding a future customer type a matter of adding one class rather than editing existing logic.
- Facade (MidnightCalculationFacade): reduces the nightly batch job to one method call, centralises the fail-safe error handling, and lets each subsystem, especially the risk-scoring service, evolve independently behind a stable interface.

6.2 Comparing Their Roles

Pattern	Category	Solves	Use it when
Singleton	Creational	Controlling how many instances of a class exist	Exactly one shared instance must exist system-wide, such as shared config, a logger, or a connection pool.
Factory Method	Creational	Decoupling object creation from the code that uses the object	A class cannot know in advance which concrete subclass it needs, or new variants will be added over time.
Facade	Structural	Simplifying access to a complex set of collaborating classes	A subsystem has many moving parts, but callers only ever need one coordinated outcome from it.

Together, they address three different concerns. Singleton controls instance count, Factory Method controls which class gets instantiated, and Facade controls how a group of already-instantiated classes is exposed to a caller. In this system, the Factory Method produces objects that could be used inside a Facade, and a Facade could just as easily read from a Singleton config. The three patterns are complementary rather than competing choices.

---

7. Generative AI Usage

7.1 Prompts Used

1. Attached the project's SRS.md (a specification document I had already written for use with AI agents) together with the assignment guidelines, and asked the AI to read both and discuss the business project with me before drafting answers to each question.

2. Drew the initial use case diagram by hand and asked the AI to review it; the AI flagged that it was missing include and extend relationships.
3. While reviewing that feedback, I noticed my own SRS.md contained two inaccuracies: it implied customers place orders directly and that customers self-register. Neither is correct; Sales Representatives visit and onboard clients, and place orders on their behalf. After correcting this, I asked the AI to redraw the use case diagram using Mermaid.js in a self-contained HTML file.
4. For each design pattern question, I identified the scenario myself and prompted the AI with that scenario, asking it to generate the corresponding UML class diagram and Java implementation with correct syntax.
5. Midway through, the AI hallucinated a payment-gateway-style option as part of the Factory Method scenario, which does not reflect how the system works. I corrected it and clarified the actual Cash/Credit customer model.
6. Provided my draft content and asked the AI to fix grammar, format it into an academic style, and assemble it into a self-contained HTML answer sheet.

7.2 What Was AI-Assisted

The project overview draft, the review and correction of the use case diagram, the three design-pattern UML class diagrams and their Java implementations, and the final HTML formatting were all AI-assisted. The underlying project scenarios, the correction of the SRS inaccuracies, and the identification of which pattern fits which part of the system were my own.

7.3 Verification and Improvement

- Caught two inaccuracies in my own SRS.md, direct customer ordering and customer self-registration, before they could be carried into the diagrams, and corrected the source understanding first rather than letting the AI build on a wrong premise.
- Caught the AI hallucinating a payment-gateway-style option in the Factory Method scenario and corrected it to match the real Cash (COD) and Credit customer model, which does not involve any payment gateway.
- Reviewed the final draft for grammar and academic formatting before accepting the generated HTML answer sheet.